

Big Data

AI Arabic Detection

Abdulaziz Hasan Aljohani
4714236

Abstract

This project develops a scalable, distributed big data pipeline to detect AI-generated Arabic abstracts. Leveraging Apache Spark on a local cluster, the system processes the "Arabic Generated Abstracts" dataset from Hugging Face. A preprocessing module normalizes Arabic text, removes stop words, and tokenizes content. Feature engineering combines TF-IDF vectors with two custom stylometric features: the ratio of long words (f011) and total line count (f034). Three classifiers Logistic Regression, Naive Bayes, and Random Forest are trained and evaluated. Logistic Regression achieved the highest validation F1-score (0.9424) and test accuracy (93.77%). A file-based streaming simulation (mimicking a Kappa architecture) demonstrates real-time inference capabilities. The project confirms that TF-IDF combined with basic stylometry is highly effective for this binary classification task, achieving near-state-of-the-art performance on the benchmark.

Introduction

The proliferation of Large Language Models (LLMs) capable of generating fluent, human-like Arabic text presents a significant challenge. Malicious use cases, including fake news generation, academic dishonesty, and automated social media manipulation, necessitate robust detection methods.

This project is motivated by the need for a production-ready, scalable solution capable of handling large volumes of text data. The primary objectives are:

1. To design and implement a distributed data processing pipeline using Apache Spark for analyzing Arabic text.
2. To engineer scalable features (TF-IDF and handcrafted stylometrics) suitable for batch and stream processing.
3. To train and evaluate multiple classification models to distinguish human-written from AI-generated scientific abstracts.
4. To implement a prototype stream processing module to simulate real-time inference.
5. To analyze model performance and scalability.

Related Work

The detection of machine-generated text has emerged as a critical research domain, driven by the rapid advancement and widespread accessibility of large language models (LLMs). Early approaches to text provenance detection relied heavily on stylometric analysis, examining linguistic patterns such as sentence length variance, punctuation density, and part-of-speech distributions. These methods, while computationally efficient, have demonstrated limited robustness against increasingly sophisticated generative models [1]

The introduction of transformer-based architecture,

particularly GPT-2 and subsequent models, catalyzed a shift toward statistical and neural detection frameworks. Gehrmann et al. [2] proposed GLTR (Giant Language Model Test Room), a tool that visualizes the ranking of token predictions from a language model, revealing that generated text tends to consist of highly predictable tokens. This statistical fingerprinting approach has proven effective even when the generating model is unknown, though its accuracy degrades as generation techniques evolve [3].

Concurrently, the research community has raised significant concerns regarding the societal implications of undetectable synthetic text. Kreps et al. [4] demonstrated how AI-generated content could be weaponized for media manipulation, emphasizing the urgent need for robust detection systems. A comprehensive survey by Jawahar et al. [5] critically reviewed pre-2020 detection methods, categorizing them into watermarking-based, statistical, and neural approaches, while noting their limited generalizability across domains and languages.

Recent work has specifically addressed bias and fairness in detection systems. Liang et al. [1] found that GPT detectors disproportionately flag non-native English writing as AI-generated, highlighting a critical ethical consideration that Arabic text detectors must also address. This finding underscores the importance of evaluating detection systems on diverse linguistic datasets.

The rapid growth in computational requirements, driven by the overwhelming volume of textual information, has necessitated a paradigm shift in Natural Language Processing (NLP) architectures. This paper addresses the challenge of scaling text analysis to process large document streams within reasonable time frames. The authors present a distributed architecture leveraging Apache Storm to run a complete chain of linguistic processors across multiple virtual machines. Through a series of experiments on medium and large-scale document collections, the study evaluates the effectiveness and efficiency of this streaming-based parallel processing pipeline. The experimental results

demonstrate significant performance improvements achievable through parallel architectures, suggesting that even greater gains are possible with larger clusters containing many processing nodes. This work provides foundational insights for building scalable, real-time NLP systems—a core requirement for applications like the AI-generated text detection pipeline described in this project[6].

From an architectural perspective, the need for real-time text analysis in production environments has led to the adoption of Big Data stream processing paradigms. The Kappa architecture, in particular, treats data streams as a single logical pipeline, simplifying the implementation of real-time inference systems compared to the batch-and-stream hybrid Lambda architecture.

The Lambda architecture stands as one of the foundational paradigms in Big Data system design, yet like any architectural pattern, it has both advocates and critics. This chapter provides a practical framework for determining whether Lambda is appropriate for a given use case. The author argues that the decision depends primarily on the architecture's applicability to the specific environment, noting that Lambda is not suitable for all Big Data scenarios. Alternative architectures, including the Kappa architecture, which treats streams as the single source of truth, and Fast Data architectures designed for ultra-low-latency processing, may be more appropriate depending on requirements. The chapter also acknowledges that some environments may not require an architecture-based approach at all, suggesting a simpler strategy instead: start with a data reservoir and design analytics incrementally as needs evolve. The core recommendation is to recognize that every environment has unique characteristics requiring special design considerations, but the optimal starting point is a generic, well-understood approach the one that most closely aligns with the organization's data velocity, volume, and variety requirements.[7]. File-based and message-queue-based stream processing with Apache Spark Structured Streaming has emerged as a standard pattern for low-latency text classification tasks [8].

For the Arabic language specifically, the availability of benchmark datasets remains limited. The KFUPM-JRCAI/arabic-generated-abstracts dataset [9] represents a significant contribution, providing multi-LLM-generated abstracts aligned with human-written scientific content. Prior work on Arabic text detection primarily focused on plagiarism or authorship attribution rather than LLM-generated content, making this dataset particularly valuable for advancing the field.

Dataset Description

Source: The dataset is KFUPM-JRCAL/arabic-generated-abstracts loaded from Hugging Face.

Size: The raw data contains three splits (by_polishing, from_title, from_title_and_content). The notebook reshapes the first split (by_polishing) into a long format, resulting in 14,255 total samples.

Class Distribution: The dataset is imbalanced.

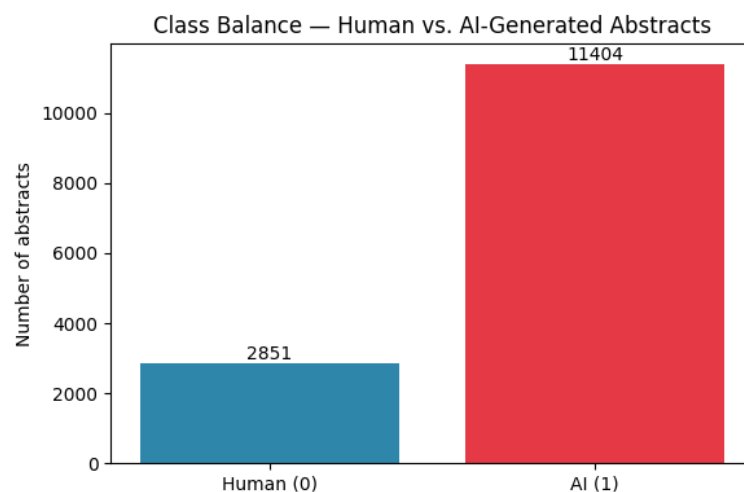
- **Human (Label 0):** 2,851 samples (20%).
- **AI-Generated (Label 1):** 11,404 samples (80%).
- The AI samples are balanced across four models: Allam, Jais, Llama, and OpenAI (2,851 samples each).

Sample Samples (from code output):

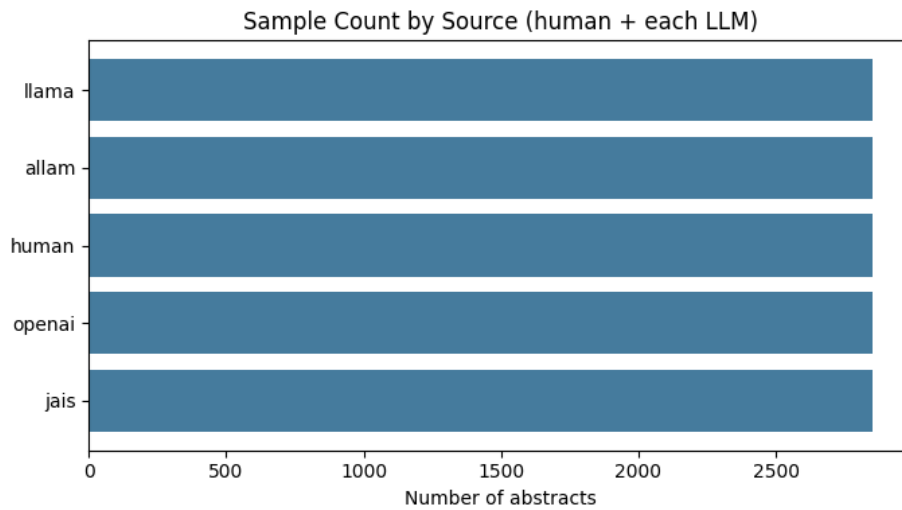
- **Human:** "...كثيرا ما ارتبطت المصادر التاريخية في الأندلس خ"
- **AI (Allam):** "...يتناول هذا البحث موضوع التعليم بين النساء الآن"
- **AI (Jais):** "...تدرس هذه الدراسة دور المرأة في التعليم في الآن"

Exploratory Data Analysis

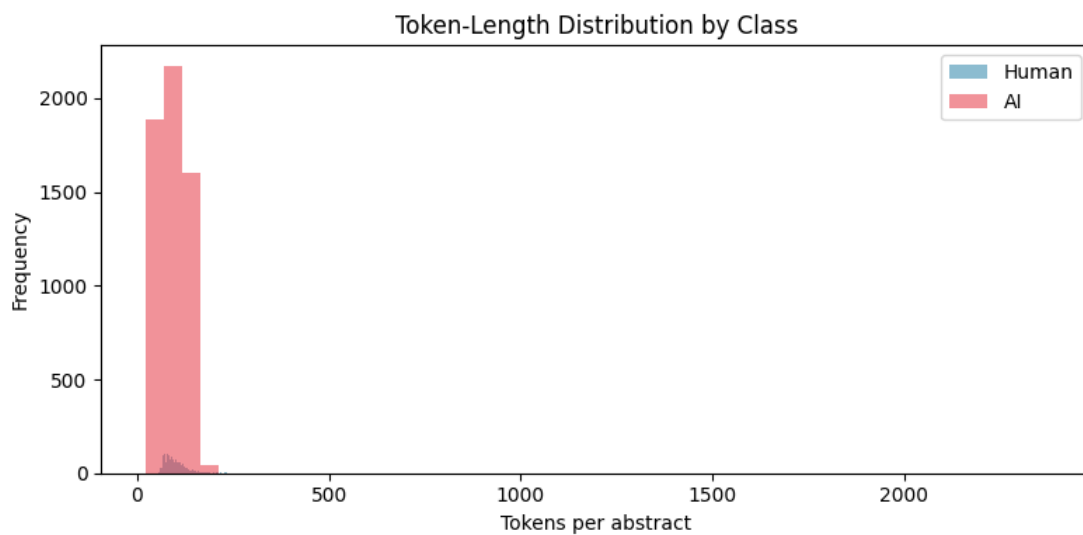
1- Class Balance



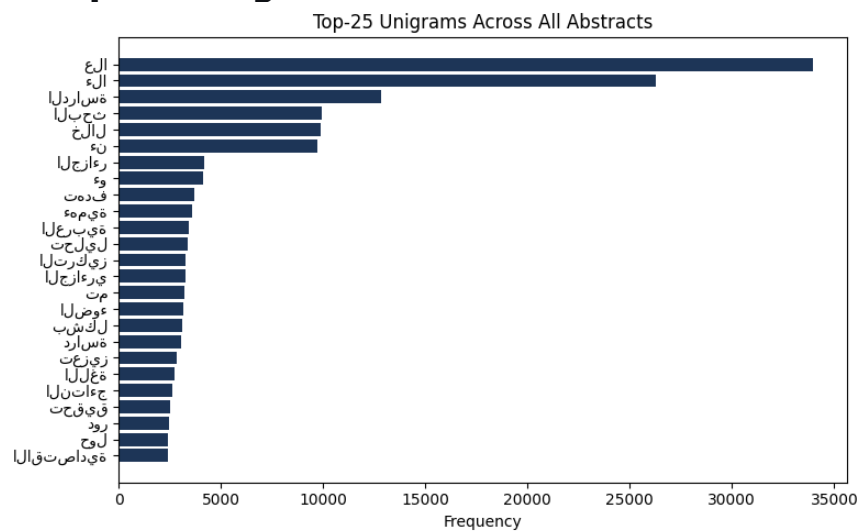
2-Sample count by source :



3-Token-Length Distribution by Class



4-Top-25 Unigrams Across All Abstracts



Methodology

The methodology is divided into four phases, mirroring the notebook's structure.

1. Environment Setup

- A local Spark session (`master("local[*]")`) is initialized with optimized memory and shuffle partitions.
- The dataset is loaded, flattened (melted) from a wide to a long format, and loaded into a Spark DataFrame.

2. Preprocessing and Feature Engineering:

- **Arabic Normalization:** A UDF (`_normalize_arabic`) uses `pyarabic.araby` to remove diacritics, unify Hamza/Alef, and strip non-Arabic characters.
- **Tokenization & Stopwords:** NLTK's Arabic stopwords list (701 words) is broadcasted. A UDF tokenizes text and filters out stopwords and short tokens (<2 chars).
- **Persistent Storage:** The cleaned data is saved as Parquet (`data/processed/abstracts.parquet`) for efficient columnar storage.

Stylometric Features:

- `f011 (num_long_words_over_N)`: Ratio of words longer than 6 characters to total word count.
- `f034 (total_lines)`: Physical line count in the original text.

TF-IDF: HashingTF (4,096 features) and IDF are applied to the tokens column.

Final Feature Vector: Assembled from the TF-IDF vector and the normalized stylometric features (stylo).

3. Batch Modeling:

- Data split: 70% training, 15% validation, 15% test.
- Models Trained: Logistic Regression, Multinomial Naive Bayes, Random Forest (60 trees).
- Evaluation Metrics: Accuracy, F1-score, Area Under the ROC Curve (AUC).
- Best Model: Logistic Regression was selected based on validation F1-score.
- Final Test Evaluation: The best model is evaluated on the held-out test set, generating a confusion matrix.

4. Stream Processing (Kappa-like Architecture):

- Simulates a file-based stream source (JSON files dropped into a directory).
- Uses Spark Structured Streaming to read the files, apply the same preprocessing and feature engineering pipeline, and run inference using the pre-trained Logistic Regression model.
- Predictions are written as JSON to an output directory for downstream consumption.

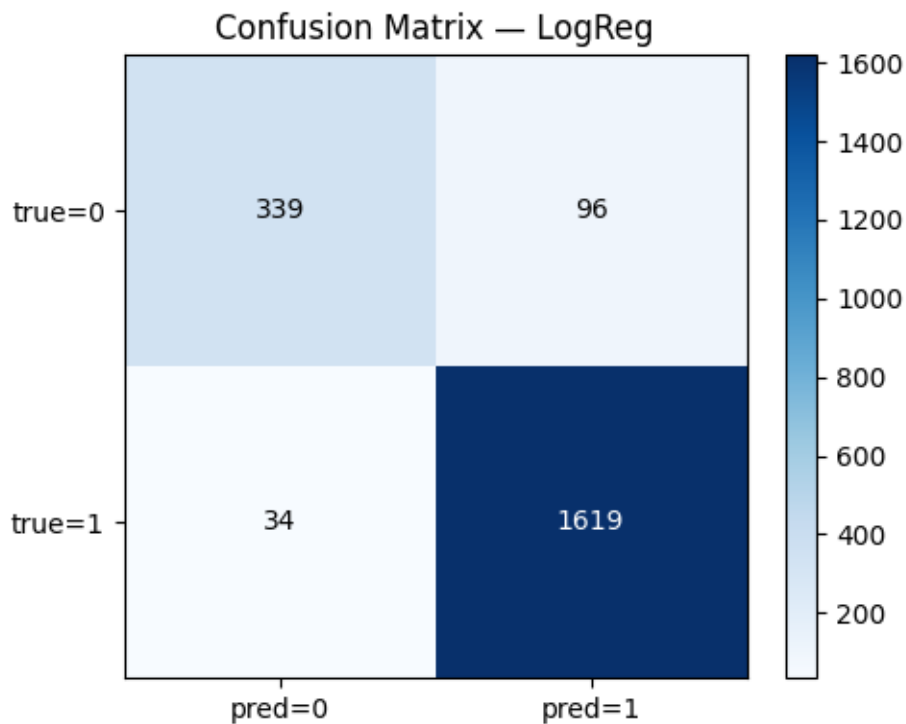
Results

- **Model Performance (Validation Set):**

Model	Accuracy	F1-Score	AUC	Training Time (s)
Logistic Regression	0.9435	0.9424	0.9786	8.75
Naive Bayes	0.8949	0.9009	0.6229	2.15
Random Forest	0.8130	0.7310	0.9706	13.19

Test Set Performance (Logistic Regression):

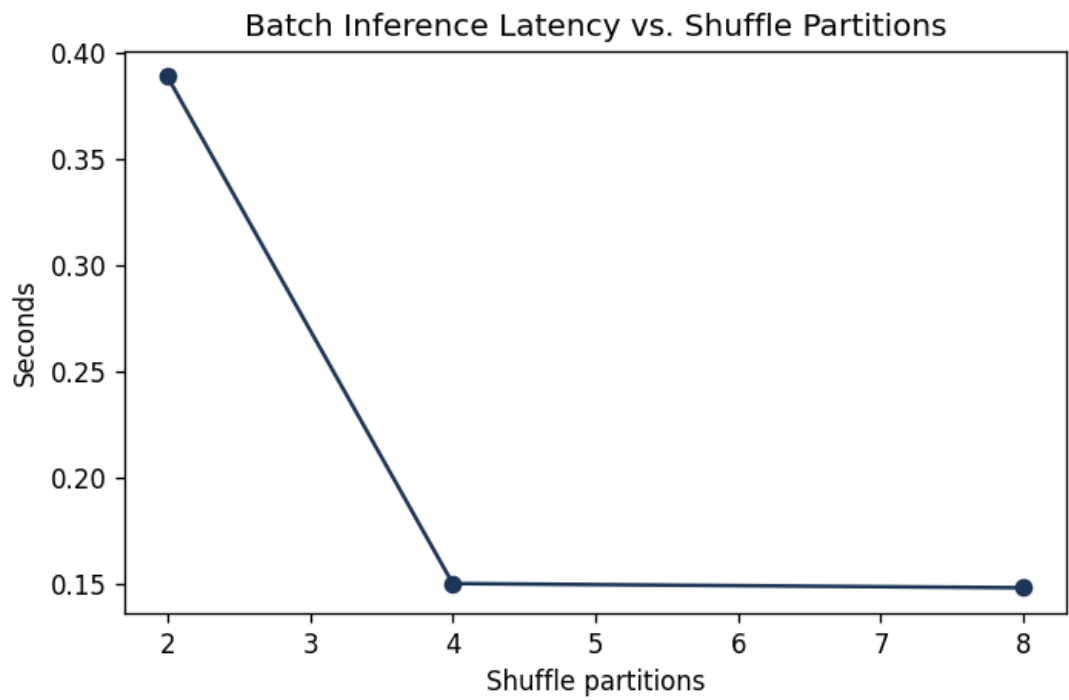
- **Accuracy:** 0.9377
- **F1-Score:** 0.9359
- **AUC:** 0.9767
- **Confusion Matrix for the best models performance:**



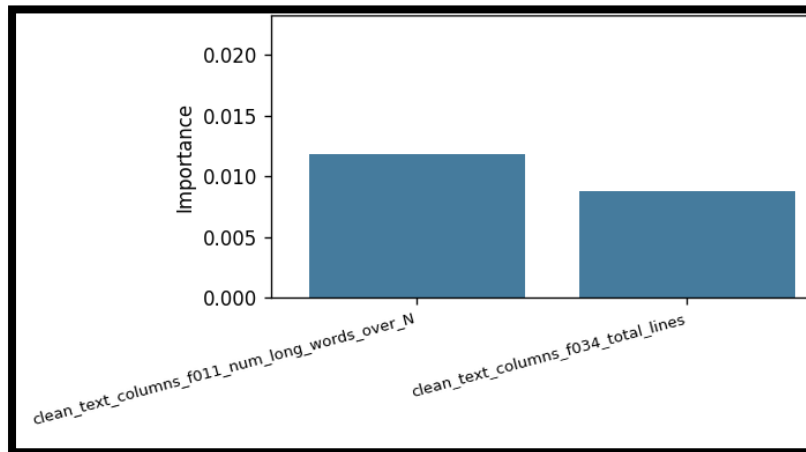
- **Analysis:** The model shows high precision for AI detection (1619 correct vs. 34 false positives). It struggles more with false negatives (96 human samples misclassified as AI), indicating a slight bias towards the majority class, which is common in imbalanced datasets.
- **Streaming Simulation:**
 - The streaming query successfully processed 20 simulated JSON files.
 - It achieved 100% accuracy on this small, held-out test batch.

Scalability Benchmark:

- Inference time on the test set (2088 samples) decreased as shuffle partitions increased.
- 2 partitions: 0.389 seconds
- 4 partitions: 0.150 seconds
- 8 partitions: 0.148 seconds
- Conclusion: The pipeline scales effectively with parallelism, though returns diminish after 4 partitions for this dataset size.



Feature Importance:



Conclusion & Future Work

Conclusion:

This project apply a complete, scalable big data pipeline for Arabic AI-text detection. The Logistic Regression model, trained on a combination of TF-IDF and simple stylometric features, achieved strong performance ($F1 > 0.94$) despite the dataset's imbalance. The implementation of a file-based stream processor validates the architecture's readiness for real-time or near-real-time applications in a production environment.

Limitations:

1. **Narrow Domain:** The model was trained only on scientific abstracts. Its performance on other genres (e.g., social media, news headlines, creative writing) is unknown.
2. **Basic Features:** While effective, the features are simplistic. The approach may fail against more sophisticated, human-like LLMs.

Future Work:

1. **Advanced Features:** Integrate more linguistic features (e.g., POS tagging, n-gram perplexity from a small LM) or fine-tune a small transformer (e.g., AraBERT) as a feature extractor.
2. **Explainability:** Use SHAP or LIME on the Spark model to explain individual predictions, building user trust in the "black box."
3. **Multi-label Classification:** Extend the problem to classify which LLM generated the text (e.g., Allam vs. Jais) for fine-grained attribution.

References

- [1] Liang, W., Yuksekogonul, M., Mao, Y., Wu, E., & Zou, J. (2023). GPT detectors are biased against non-native English writers. *Patterns*, 4(7).
- [2] Gehrmann, S., Strobelt, H., & Rush, A. M. (2019, July). Gltr: Statistical detection and visualization of generated text. In *Proceedings of the 57th annual meeting of the association for computational linguistics: system demonstrations* (pp. 111-116).
- [3] Ippolito, D., Duckworth, D., Callison-Burch, C., & Eck, D. (2020, July). Automatic detection of generated text is easiest when humans are fooled. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 1808-1822).
- [4] Jiang, M. (2020). Antiferromagnetism mediated by heavy electrons: singlet shielding vs" high-order" RKKY. arXiv preprint arXiv:2005.14148.
- [5] Jawahar, G., Abdul-Mageed, M., & Laks Lakshmanan, V. S. (2020, December). Automatic detection of machine generated text: A critical survey. In *Proceedings of the 28th international conference on computational linguistics* (pp. 2296-2309).
- [6] Agerri, R., Artola, X., Beloki, Z., Rigau, G., & Soroa, A. (2015). Big data for Natural Language Processing: A streaming approach. *Knowledge-Based Systems*, 79, 36-42.
- [7] Lakhe, B. (2016). Case Study: Implementing Lambda Architecture. In: *Practical Hadoop Migration*. Apress, Berkeley, CA.
https://doi.org/10.1007/978-1-4842-1287-5_11
- [8] Armbrust, M., Das, T., Torres, J., Yavuz, B., Zhu, S., Xin, R., ... & Zaharia, M. (2018, May). Structured streaming: A declarative api for real-time applications in apache spark. In *Proceedings of the 2018 International Conference on Management of Data* (pp. 601-613)..
- [9] KFUPM-JRCAI, "Arabic generated abstracts," *Hugging Face Datasets*, 2023. [Online]. Available: <https://huggingface.co/datasets/KFUPM-JRCAI/arabic-generated-abstracts>